Unit 4: ReactJS — Easy Explanation

This PPT explains five important React topics:

1. Routing
2. CSS styling
3. Hooks: `useState` and `useEffect`
4. Forms
5. Consuming APIs and CRUD operations

The examples below are based on your uploaded PPT. cite file:1

1. Routing in React

What is routing?

Routing means showing different components when the URL changes.

Example:

URL	Component
`/`	Home
`/about`	About
`/contact`	Contact

In a normal website, each URL may load a new HTML page. In React, routing changes the displayed component without fully reloading the page.

Install React Router

```
npm install react-router-dom
```

This package provides routing features for React. cite file:1

Basic folder structure

```
src/
  pages/
    Layout.js
    Home.js
    About.js
    Contact.js
  index.js
```

`index.js`

```
import React from "react";
import ReactDOM from "react-dom/client";

import {
  BrowserRouter,
  Routes,
  Route
} from "react-router-dom";

import Layout from "./pages/Layout";
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";

const root = ReactDOM.createRoot(
  document.getElementById("root")
);

root.render(
  <BrowserRouter>
    <Routes>
      <Route path="/" element={<Layout />}>
        <Route index element={<Home />} />
        <Route path="about" element={<About />} />
        <Route path="contact" element={<Contact />} />
      </Route>
    </Routes>
  </BrowserRouter>
);
```

Line-by-line explanation

```
import React from "react";
```

Imports React.

```
import ReactDOM from "react-dom/client";
```

Used to display the React application in the browser.

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

- `BrowserRouter`: Enables routing.
- `Routes`: Holds all routes.
- `Route`: Defines one URL and its component.

```
import Home from "./pages/Home";
```

Imports the `Home` component.

```
const root = ReactDOM.createRoot(
  document.getElementById("root")
);
```

Finds the HTML element with ID `root`.

```
root.render(...);
```

Displays React content.

```
<BrowserRouter>
```

Starts routing support.

```
<Routes>
```

Contains route definitions.

```
<Route path="/" element={<Layout />}>
```

When the URL starts with `/`, React displays `Layout`.

```
<Route index element={<Home />} />
```

The default page is `Home`.

```
<Route path="about" element={<About />} />
```

The URL `/about` displays `About`.

```
<Route path="contact" element={<Contact />} />
```

The URL `/contact` displays `Contact`.

Page components

`Home.js`

```
function Home() {
  return <h1>Home Page</h1>;
}

export default Home;
```

- Creates a component called `Home`.
- Returns an `<h1>` heading.
- Exports the component.

`About.js`

```
function About() {
  return <h1>About Page</h1>;
}

export default About;
```

`Contact.js`

```
function Contact() {
  return <h1>Contact Page</h1>;
}

export default Contact;
```

Navigation using `Link`

```
import { Link, Outlet } from "react-router-dom";

function Layout() {
  return (
    <>
      <h1>Header</h1>

      <ul>
        <li>
          <Link to="/">Home</Link>
```

```
        </li>

        <li>
          <Link to="/about">About</Link>
        </li>

        <li>
          <Link to="/contact">Contact</Link>
        </li>
      </ul>

      <Outlet />

      <h1>Footer</h1>
    </>
  );
}

export default Layout;
```

Important lines

```
<Link to="/about">About</Link>
```

Creates a navigation link to `/about`.

```
<Outlet />
```

Displays the child route component.

For example:

- `Layout` displays the header and footer.
- `<Outlet />` displays `Home`, `About`, or `Contact`.

Simple example

If the URL is:

```
http://localhost:3000/about
```

React displays:

```
Header
Home | About | Contact
About Page
Footer
```

2. CSS Styling in React

The PPT explains three common CSS methods:

1. Inline styling.
2. External CSS stylesheet.
3. CSS Modules. 【cite】file:1】

Method 1: Inline styling

```
function Home() {
  return (
    <h1
      style={{
        color: "red",
        backgroundColor: "blue"
      }}
    >
      Hello World
    </h1>
  );
}
```

Explanation

```
style={{ ... }}
```

There are two curly brackets:

- Outer `{}`: JavaScript expression inside JSX.
- Inner `{}`: JavaScript object.

```
color: "red"
```

Changes text color.

```
backgroundColor: "blue"
```

Changes background color.

In JSX, CSS properties use camelCase:

```
background-color
```

becomes:

```
backgroundColor
```

You can also create a separate object:

```
const styleObj = {
  color: "red",
  backgroundColor: "blue"
};

function Home() {
  return <h1 style={styleObj}>Hello World</h1>;
}
```

Method 2: External CSS

`Style.css`

```
.myClass {
  color: red;
  background-color: blue;
}
```

`Home.js`

```
import "./Style.css";

function Home() {
  return (
    <div className="myClass">
      Hello World
    </div>
  );
}

export default Home;
```

Explanation

```
import "./Style.css";
```

Loads the CSS file.

```
className="myClass"
```

Connects the HTML element to the CSS class.

In React, use `className`, not `class`.

Method 3: CSS Modules

CSS Modules prevent class-name conflicts between components.

`mystyle.module.css`

```
.myClass {
  color: red;
  background-color: blue;
}
```

`Home.js`

```
import styles from "./mystyle.module.css";

function Home() {
  return (
    <div className={styles.myClass}>
      Hello World
    </div>
  );
}

export default Home;
```

Explanation

```
import styles from "./mystyle.module.css";
```

Imports the CSS module as an object.

```
className={styles.myClass}
```

Uses the `myClass` style from that module.

CSS Modules automatically create unique class names.

3. React Hooks

What is a Hook?

A Hook is a React function that allows function components to use React features such as state and lifecycle behavior.

Hooks were introduced in React 16.8. They allow us to write React code without class components. 【cite】file:1】

Important Hooks include:

- `useState`
- `useEffect`
- `useContext`
- `useRef`
- `useReducer`
- `useCallback`
- `useMemo`

Rules of Hooks

- Call Hooks only inside React function components.
- Call Hooks at the top level.
- Do not call Hooks inside `if`, `for`, or nested functions.

Incorrect:

```
if (isLoggedIn) {
  const [name, setName] = useState("");
}
```

Correct:

```
const [name, setName] = useState("");

if (isLoggedIn) {
  console.log(name);
}
```

4. `useState` Hook

What is state?

State is data that can change during the execution of a React application.

Examples:

- Counter value.
- Username.
- Login status.
- List of products.
- Form data.

Basic example

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <>
      <h1>{count}</h1>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </>
  );
}

export default Counter;
```

Line-by-line explanation

```
import { useState } from "react";
```

Imports the `useState` Hook.

```
const [count, setCount] = useState(0);
```

This returns two values:

- `count`: Current value.
- `setCount`: Function used to update the value.

```
useState(0)
```

Sets the initial value to `0`.

```
<h1>{count}</h1>
```

Displays the current count.

```
setCount(count + 1)
```

Increases the count by one.

Important rule

Correct:

```
setCount(20);
```

Incorrect:

```
count = 20;
```

React state must be changed using the setter function.

How it works

Initial state:

```
count = 0
```

After clicking the button:

```
count = 1
```

React automatically renders the component again.

5. `useEffect` Hook

What is `useEffect`?

`useEffect` performs side effects.

Examples of side effects:

- Calling an API.
- Starting a timer.

- Updating the browser title.
- Reading data from local storage.
- Adding event listeners.

The syntax is:

```
useEffect(function, dependencyArray);
```

Example 1: Run after every render

```
useEffect(() => {
  console.log("Component rendered");
});
```

Because there is no dependency array, it runs after every render.

Example 2: Run only once

```
useEffect(() => {
  console.log("Component loaded");
}, []);
```

The empty array means it runs only after the first render.

Example 3: Run when a value changes

```
useEffect(() => {
  console.log("Count changed");
}, [count]);
```

This runs whenever `count` changes.

Easy table

Code	When it runs
`useEffect(() => {})`	After every render
`useEffect(() => {}, [])`	Once
`useEffect(() => {}, [count])`	When `count` changes

6. Forms in React

HTML forms normally manage their own input values. React forms usually store input values in state. This is called a controlled component. 【cite】file:1】

Basic controlled form

```
import { useState } from "react";

function MyForm() {
  const [name, setName] = useState("");

  function handleSubmit(event) {
    event.preventDefault();
    console.log(name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={name}
        onChange={(event) => {
          setName(event.target.value);
        }}
      />

      <input type="submit" value="Submit" />
    </form>
  );
}

export default MyForm;
```

Explanation

```
const [name, setName] = useState("");
```

Stores the input value.

```
value={name}
```

The input displays the value from React state.

```
onChange={(event) => {
  setName(event.target.value);
}}
```

Every time the user types, React updates the state.

```
event.target.value
```

Gets the current input value.

```
onSubmit={handleSubmit}
```

Calls `handleSubmit` when the form is submitted.

```
event.preventDefault();
```

Stops the browser from refreshing or opening another page.

```
console.log(name);
```

Displays the entered name in the browser console.

7. Handling Multiple Inputs

```
import { useState } from "react";

function MyForm() {
  const [data, setData] = useState({
    userName: "",
    password: ""
  });

  function handleSubmit(event) {
    event.preventDefault();

    console.log(data.userName);
    console.log(data.password);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={data.userName}
        onChange={(event) => {
          setData({
            ...data,
            userName: event.target.value
          });
        }}
      />

      <input
        type="password"
        value={data.password}
```

```
        onChange={(event) => {
          setData({
            ...data,
            password: event.target.value
          });
        }}
      />

      <input type="submit" value="Login" />
    </form>
  );
}
```

Important line

```
setData({
  ...data,
  userName: event.target.value
});
```

Explanation:

```
...data
```

Copies the old values.

```
userName: event.target.value
```

Changes only the username.

Without `...data`, the password value could be removed.

8. Consuming an API

What is an API?

An API allows one application to communicate with another application.

Example:

```
React application → API server → Database
```

Suppose the API returns faculty data:

```
[
  {
    "id": "1",
```

```
    "FacultyName": "Rahul Patel",
    "FacultyDepartment": "Computer Science",
    "FacultySalary": 50000
  }
]
```

React can fetch this data and display it.

9. Mock API

A mock API is a temporary fake API used for development and practice.

The PPT uses `mockapi.io`. You can:

1. Create an account.
2. Create a project.
3. Add a resource such as `faculties`.
4. Add fields.
5. Generate sample records.
6. Copy the API endpoint.

The endpoint may look like:

```
https://your-project.mockapi.io/api/v1/faculties
```

The API supports CRUD operations:

Operation	HTTP method
Get all records	`GET`
Get one record	`GET /id`
Add record	`POST`
Update record	`PUT`
Delete record	`DELETE`

10. Get All Records

```
import { useEffect, useState } from "react";

function Faculties() {
  const [faculties, setFaculties] = useState([]);

  useEffect(() => {
    const apiUrl =
      "https://your-project.mockapi.io/api/v1/faculties";
```

```
    fetch(apiUrl)
      .then((response) => response.json())
      .then((result) => {
        setFaculties(result);
      });
  }, []);

  return (
    <div>
      {faculties.map((faculty) => (
        <div key={faculty.id}>
          <h3>{faculty.FacultyName}</h3>
          <p>{faculty.FacultyDepartment}</p>
        </div>
      ))}
    </div>
  );
}

export default Faculties;
```

Line-by-line explanation

```
const [faculties, setFaculties] = useState([]);
```

Creates an empty array for faculty records.

```
useEffect(() => {
```

Runs code after the component loads.

```
fetch(apiUrl)
```

Sends a request to the API.

```
response.json()
```

Converts the response into JavaScript data.

```
setFaculties(result)
```

Stores the API data in state.

```
}, []);
```

Runs the API request only once.

```
faculties.map(...)
```

Loops through every faculty.

```
key={faculty.id}
```

Gives each React element a unique key.

```
{faculty.FacultyName}
```

Displays the faculty name.

11. Get One Record by ID

Suppose the URL is:

```
/faculties/5
```

The `5` is the faculty ID.

```
import {
  useParams,
  useNavigate
} from "react-router-dom";

import { useEffect, useState } from "react";

function FacultyDetail() {
  const { id } = useParams();
  const [faculty, setFaculty] = useState({});

  useEffect(() => {
    const apiUrl =
      "https://your-project.mockapi.io/api/v1/faculties";

    fetch(`${apiUrl}/${id}`)
      .then((response) => response.json())
      .then((result) => {
        setFaculty(result);
      });
  }, [id]);

  return (
    <div>
      <h1>{faculty.FacultyName}</h1>
      <p>{faculty.FacultyDepartment}</p>
      <p>{faculty.FacultySalary}</p>
      <img src={faculty.FacultyImage} alt="Faculty" />
    </div>
```

```
  );
}

export default FacultyDetail;
```

Important lines

```
const { id } = useParams();
```

Reads the ID from the URL.

```
fetch(`${apiUrl}/${id}`)
```

Creates a URL such as:

```
https://your-project.mockapi.io/api/v1/faculties/5
```

```
useEffect(..., [id])
```

Runs the request when the ID changes.

Route

```
<Route
  path="/faculties/:id"
  element={<FacultyDetail />}
/>
```

`:id` is a dynamic route parameter.

12. Delete a Record

```
function deleteFaculty(id) {
  fetch(`${apiUrl}/${id}`, {
    method: "DELETE"
  })
    .then(() => {
      navigate("/faculties");
    });
}
```

Explanation

```
method: "DELETE"
```

Tells the API to delete the record.

```
navigate("/faculties")
```

Returns the user to the faculty list after deletion.

Example button:

```
<button onClick={() => deleteFaculty(faculty.id)}>
  Delete
</button>
```

13. Insert a Record

```
function AddFaculty() {
  const navigate = useNavigate();

  const apiUrl =
    "https://your-project.mockapi.io/api/v1/faculties";

  const [data, setData] = useState({
    FacultyName: "",
    FacultyDepartment: "",
    FacultySalary: ""
  });

  function addFaculty() {
    fetch(apiUrl, {
      method: "POST",
      body: JSON.stringify(data),
      headers: {
        "Content-Type": "application/json"
      }
    })
      .then(() => {
        navigate("/faculties");
      });
  }

  return (
    <div>
      <input
        type="text"
        placeholder="Faculty name"
        value={data.FacultyName}
        onChange={(event) => {
```

```
            setData({
              ...data,
              FacultyName: event.target.value
            });
          }}
        />

        <input
          type="text"
          placeholder="Department"
          value={data.FacultyDepartment}
          onChange={(event) => {
            setData({
              ...data,
              FacultyDepartment: event.target.value
            });
          }}
        />

        <input
          type="number"
          placeholder="Salary"
          value={data.FacultySalary}
          onChange={(event) => {
            setData({
              ...data,
              FacultySalary: event.target.value
            });
          }}
        />

        <button onClick={addFaculty}>
          Add Faculty
        </button>
      </div>
    );
}
```

Important lines

```
method: "POST"
```

Creates a new record.

```
JSON.stringify(data)
```

Converts JavaScript data into JSON text.

```
"Content-Type": "application/json"
```

Tells the server that the request body is JSON.

```
navigate("/faculties")
```

Returns to the list page after adding the faculty.

14. Update a Record

```
function updateFaculty(id, data) {
  fetch(`${apiUrl}/${id}`, {
    method: "PUT",
    body: JSON.stringify(data),
    headers: {
      "Content-Type": "application/json"
    }
  })
    .then(() => {
      navigate("/faculties");
    });
}
```

Difference between POST and PUT

Method	Purpose
POST	Add a new record
PUT	Update an existing record

The URL must include the record ID:

```
/faculties/5
```

Complete Working Flow

Suppose you build a Faculty Management application.

Step 1: Display all faculties

```
/faculties
```

React sends a `GET` request and displays all records.

Step 2: Open one faculty

```
/faculties/3
```

React reads ID `3`, calls the API, and displays one record.

Step 3: Add a faculty

```
/faculties/add
```

The user fills out a form. React sends a `POST` request.

Step 4: Edit a faculty

```
/faculties/edit/3
```

React first loads faculty `3`, then sends a `PUT` request after editing.

Step 5: Delete a faculty

The user clicks Delete. React sends a `DELETE` request.

Very Short Revision

Topic	Meaning
Routing	Display components according to URL
`BrowserRouter`	Enables routing
`Routes`	Holds route definitions
`Route`	Connects URL to component
`Link`	Navigates without page reload
`Outlet`	Displays child route
Inline CSS	CSS written directly in JSX
CSS file	CSS written in a separate file
CSS Module	Component-specific CSS
`useState`	Stores changing data
`useEffect`	Performs side effects
Controlled form	Input value controlled by React state
`fetch()`	Sends API request

Topic	Meaning
`GET`	Reads data
`POST`	Creates data
`PUT`	Updates data
`DELETE`	Removes data
`useParams`	Reads values from URL
`useNavigate`	Moves to another route
`map()`	Loops through an array

Most important idea

A React API application usually follows this pattern:

```
useState → store data
useEffect → call API
fetch → communicate with server
map → display many records
useParams → read ID
useNavigate → change page
```

For example:

```
const [data, setData] = useState([]);

useEffect(() => {
  fetch(apiUrl)
    .then((response) => response.json())
    .then((result) => setData(result));
}, []);

return data.map((item) => (
  <p key={item.id}>{item.name}</p>
));
```

This means:

1. Create empty state.
2. Call the API.
3. Convert response to JSON.
4. Store response in state.
5. Loop over the data.
6. Display each item.

⁂

1. unit-4.pptx